

Session 7.2: Error Handling - Robust Programs

Handle Errors Gracefully • Week 7, Day 2

Today's Journey

Part 1: Exceptions & Try/Except

Part 2: Workshop - Handle Common Errors

Break

Part 3: Raising Exceptions & Debugging

Part 4: Workshop - Build Robust Programs

Part 5: Quiz & Week 7 Wrap-up

Part 1: Exceptions

Handle errors gracefully!

Last Session Review

Session 7.1 you learned:

- ✓ Reading files (with statement)
- ✓ Writing files (modes w, a, x)
- ✓ CSV reading/writing
- ✓ File paths

Today: Error Handling!

What happens when things go wrong?

- File doesn't exist
- Invalid input
- Network failures
- Division by zero

Professional error handling!

What Are Exceptions?

Exception = error that occurs during execution

Without handling:

- Program crashes
- User sees ugly error
- Data might be lost

With handling:

- Program continues
- User sees friendly message
- Can retry or recover

Professional programs don't crash!

They handle errors gracefully!

Common Exception Types

Python has many exception types:

- `ValueError` - Invalid value type
- `TypeError` - Wrong type
- `FileNotFoundError` - File doesn't exist
- `ZeroDivisionError` - Divide by zero
- `KeyError` - Dict key doesn't exist
- `IndexError` - List index out of range
- `AttributeError` - Attribute doesn't exist

Each represents different error!

What Happens Without Error Handling

```
# Code that crashes
number = int('abc')    # ValueError!

# Program stops here!
print('This never runs')
# Error message:
# ValueError: invalid literal for int() with base 10: 'abc'
# Traceback (most recent call last):
#   File ...

# User sees scary error
# Program terminates
# Bad experience!

# We need to handle this!
```

Try/Except - Basic Pattern

try/except = error handling block

Format:

try:

 # risky code

except ExceptionType:

 # handle error

Flow:

1. Try the risky code
2. If error occurs, catch it
3. Run except block instead of crashing
4. Continue program

Program stays running!

Your First Try/Except

```
# Handle invalid input
try:
    age = int(input('Enter your age: '))
    print(f'You are {age} years old')
except ValueError:
    print('Error: Please enter a valid number')
print('Program continues!')
```

```
# If user enters 'abc':
# Error: Please enter a valid number
# Program continues!
```

```
# If user enters '25':
# You are 25 years old
# Program continues!
# No crash either way!
```

Catching Multiple Exception Types

Can catch different exceptions:

Method 1: Multiple except blocks

try:

code

except ValueError:

handle value error

except TypeError:

handle type error

Method 2: Catch multiple at once

try:

code

except (ValueError, TypeError):

handle both

Handle each error appropriately!

Multiple Exceptions Example

```
# Risky calculation
try:
    num1 = int(input('First number: '))
    num2 = int(input('Second number: '))
    result = num1 / num2
    print(f'Result: {result}')

except ValueError:
    print('Error: Please enter valid numbers')

except ZeroDivisionError:
    print('Error: Cannot divide by zero')

except Exception as e:
    print(f'Unexpected error: {e}')
# Each error type handled differently!
# Last except catches anything else
```

Try/Except/Else/Finally

Full error handling pattern:

try:

 # risky code

except:

 # if error occurs

else:

 # if NO error (optional)

finally:

 # ALWAYS runs (optional)

finally = cleanup code

- Close files
- Release resources
- Save data

Runs even if exception occurs!

Complete Try/Except/Else/Finally

```
# File processing with complete error handling
try:
    file = open('data.txt', 'r')
    content = file.read()
    print('File read successfully')
except FileNotFoundError:
    print('Error: File not found')
    content = None
else:
    # Runs only if no exception
    print(f'Loaded {len(content)} characters')
finally:
    # ALWAYS runs
    try:
        file.close()
        print('File closed')
    except:
        print('No file to close')
# Proper cleanup guaranteed!
```

Error Handling Best Practices

✓ DO:

- Catch specific exceptions
- Provide helpful error messages
- Log errors for debugging
- Clean up resources (finally)
- Fail gracefully

✗ DON'T:

- Catch Exception blindly (too broad)
- Hide errors silently (pass)
- Ignore cleanup
- Show technical errors to users

Professional error handling!

Part 2: Workshop

Handle Common Errors!

Exercise 1: Safe Input Handler

Create file: safe_input.py

Build robust input function:

```
def get_integer(prompt):  
    while True:  
        try:  
            value = int(input(prompt))  
            return value  
        except ValueError:  
            print('Invalid! Enter a number.')
```

Test:

```
age = get_integer('Age: ')
```

```
# Keeps asking until valid number
```

Exercise 2: Safe File Reader

Create file: `safe_file_reader.py`

Build robust file reader:

```
def read_file_safe(filename):  
    try:  
        with open(filename, 'r') as f:  
            return f.read()  
    except FileNotFoundError:  
        print(f'Error: {filename} not found')  
        return None  
    except PermissionError:  
        print(f'Error: No permission to read {filename}')  
        return None
```

Test with existing and non-existing files

Exercise 3: Safe Calculator

Create file: safe_calculator.py

Build calculator with error handling:

```
def safe_divide(a, b):  
    try:  
        return a / b  
    except ZeroDivisionError:  
        print('Error: Division by zero')  
        return None  
    except TypeError:  
        print('Error: Invalid types for division')  
        return None
```

Test:

```
print(safe_divide(10, 2)) # 5.0  
print(safe_divide(10, 0)) # Error message, None  
print(safe_divide(10, 'a')) # Error message, None
```



Break

Part 3: Raising Exceptions

Custom error handling

Raising Your Own Exceptions

raise = trigger an exception yourself

Format:

```
raise ExceptionType('error message')
```

Why raise exceptions?

- Signal invalid input
- Enforce requirements
- Stop execution early
- Communicate errors clearly

Professional error signaling!

Raising Exceptions Example

```
# Validate age
def set_age(age):
    if not isinstance(age, int):
        raise TypeError('Age must be an integer')
    if age < 0:
        raise ValueError('Age cannot be negative')
    if age > 150:
        raise ValueError('Age too high (max 150)')
    return age

# Usage:
try:
    age = set_age(-5)
except ValueError as e:
    print(f'Error: {e}') # Age cannot be negative
# Function enforces its requirements!
```

Custom Exception Classes

Create your own exception types:

Format:

```
class CustomError(Exception):  
    pass
```

Use:

```
raise CustomError('specific message')
```

Why custom exceptions?

- More specific errors
- Domain-specific problems
- Better error categorization

Professional libraries use this!

Custom Exception Example

```
# Define custom exceptions
class InvalidDataError(Exception):
    """Raised when data validation fails"""
    pass

class ProcessingError(Exception):
    """Raised during data processing"""
    pass

# Use custom exceptions
def process_data(data):
    if data is None:
        raise InvalidDataError('Data cannot be None')
    if len(data) == 0:
        raise InvalidDataError('Data cannot be empty')
    # Process...
    return processed_data

# Clear, specific error types!
```

Debugging Techniques

Debugging = finding and fixing bugs

1. Print debugging

- Add print statements to see values

2. Python debugger (pdb)

- `import pdb; pdb.set_trace()`

3. Exception messages

- Read the traceback!

4. Logging

- Better than print for production

Professional debugging!

Print Debugging Example

```
# Function with bug
def calculate_average(numbers):
    print(f'DEBUG: Input = {numbers}')    # Debug
    total = sum(numbers)
    print(f'DEBUG: Total = {total}')      # Debug
    count = len(numbers)
    print(f'DEBUG: Count = {count}')      # Debug
    average = total / count
    print(f'DEBUG: Average = {average}')  # Debug
    return average

# Run and see all intermediate values
result = calculate_average([10, 20, 30])

# See where things go wrong!
# Remove debug prints when fixed
```

Logging - Production Debugging

Logging = better than print

```
import logging
```

Levels:

- DEBUG - detailed info
- INFO - general info
- WARNING - warning messages
- ERROR - error occurred
- CRITICAL - serious problem

Benefits:

- Control verbosity
- Save to files
- Include timestamps

Professional applications use logging!

Basic Logging Example

```
import logging
# Configure logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)

# Use logging instead of print
def process_file(filename):
    logging.info(f'Processing {filename}')
    try:
        with open(filename, 'r') as f:
            data = f.read()
            logging.info(f'Loaded {len(data)} characters')
            return data
    except Exception as e:
        logging.error(f'Failed to process {filename}: {e}')
        return None

# Professional logging!
```

Error Handling in ML

ML projects need robust error handling:

Data loading:

```
try: load_data()  
except FileNotFoundError: ...
```

Training:

```
try: train_model()  
except MemoryError: reduce_batch_size()
```

Validation:

```
if data_invalid(): raise ValidationError()
```

Saving:

```
try: save_model()  
finally: cleanup_resources()
```

Production ML = lots of error handling!

Part 4: Workshop

Robust Programs!

Exercise 4: Robust Data Processor

Create file: robust_processor.py

Build complete error-handling system:

```
def process_student_data(filename):
```

```
    # Try to read file
```

```
    # Handle FileNotFoundError
```

```
    # Parse CSV data
```

```
    # Validate each row (try/except per row)
```

```
    # Calculate statistics
```

```
    # Log all operations
```

```
    # Return results or None
```

Requirements:

- Handle file errors
- Validate data (age 0-120, score 0-100)
- Skip invalid rows, log warnings
- Use logging module
- Graceful failure

Production-quality code!

Part 5: Quiz & Wrap-up

Week 7 Complete!

WEEK 7 COMPLETE! Professional Skills!

This week you mastered:

- ✓ File I/O (reading, writing)
- ✓ CSV processing
- ✓ Exceptions (try/except)
- ✓ Error handling patterns
- ✓ Raising exceptions
- ✓ Debugging techniques
- ✓ Logging

You can now:

- Handle errors gracefully
- Build robust programs
- Debug effectively

Production-ready skills!

Homework - Week 7 Mastery

1. Complete the quiz (10 questions)

2. Project: Robust Data Pipeline

- Build: `data_pipeline.py`
- Read CSV file with error handling
- Validate all data (skip invalid rows)
- Process and transform data
- Calculate statistics
- Write results to new CSV
- Use logging throughout
- Handle all possible errors!

3. Git Practice

- Professional commits
- Clean code

4. Prepare: Read about project organization (Week 8!)

Looking Ahead: Week 8 - Final Week!

Final week: Professional Python & ML Prep

Session 8.1: Project Organization

- Project structure
- Modules and packages
- Best practices

Session 8.2: ML Preparation & Final Project

- Python for ML overview
- NumPy and Pandas intro
- Final capstone project

Course completion!

Week 7 Reflection

Incredible week:

Monday:

- 'How do I read files?'

Today:

- 'I handle files and errors like a pro!'

You've mastered:

- Complete Python fundamentals
- File I/O and persistence
- Professional error handling

Final week next!

One week to ML-ready! See you Monday!